

WEEK 2 LEARNING GUIDE

BUILDING A SECURE SERVERLESS FEEDBACK API ON AWS

1. INTRODUCTION

The first week of the internship focused on provisioning cloud infrastructure using **Infrastructure as Code (IaC)**. Using Terraform, AWS resources such as Amazon S3, CloudFront, IAM, and backend state management were provisioned declaratively. This established the foundation required to deploy cloud infrastructure in a repeatable and automated manner.

The second week shifted the focus from infrastructure to **application development**. Rather than creating individual cloud resources, the objective was to understand how these resources can be integrated to build a complete serverless application.

Unlike traditional applications that run on continuously active servers, serverless applications are built by combining multiple managed AWS services. Compute, storage, networking, authentication, and monitoring are handled independently, allowing developers to focus primarily on implementing business logic.

The project for this week involved developing a **Feedback API** capable of collecting and retrieving user feedback through a web interface. Although the application itself was relatively simple, it introduced several important AWS services and cloud computing concepts, including:

- Serverless Computing
- AWS Lambda
- Amazon API Gateway
- Amazon Cognito
- Amazon CloudWatch
- JWT-based Authentication and Authorization
- Role-Based Access Control (RBAC)

By the end of the week, these services had been integrated into a complete serverless architecture. The frontend was hosted on Amazon S3 and distributed through CloudFront, backend logic was executed using AWS Lambda, REST APIs were exposed through API Gateway, user authentication was managed by Amazon Cognito, and the entire infrastructure continued to be provisioned using Terraform.

More importantly, this project demonstrated that modern cloud applications are rarely built around a single service. Instead, they consist of multiple specialised AWS services, each

performing a well-defined responsibility while working together to form a secure, scalable, and maintainable application.

2. CORE CONCEPTS LEARNED

SERVERLESS COMPUTING

One of the most important concepts introduced during this week was **Serverless Computing**.

Traditionally, backend applications are deployed on servers or virtual machines that remain active regardless of whether users are interacting with the application. These servers require developers to provision infrastructure, install software, configure networking, monitor performance, and scale resources as application traffic grows.

Serverless computing follows a different approach, instead of managing servers, developers focus only on writing application logic. The cloud provider automatically provisions the required infrastructure, executes the code when needed, and releases the resources once execution is complete.

Although the term *serverless* suggests that servers no longer exist, this is not entirely accurate. Servers are still responsible for executing the application. The difference is that developers no longer manage them directly.

The difference between the two approaches is illustrated below.

Traditional Application Architecture

Client

|



Web Server

|



Application

|



Database

Serverless Architecture

Client



API Gateway



AWS Lambda



Storage / Database

A serverless application operates using an **event-driven execution model**.

Instead of executing continuously, application code runs only when an event occurs. Once the required processing is complete, the execution environment is automatically released.

Event



Function Executes



Response Generated



Execution Ends

This execution model provides several important advantages.

- Automatic scaling based on incoming requests.
- Reduced infrastructure management.
- Lower operational costs through pay-per-use pricing.
- High availability without managing servers.

Another important characteristic of serverless applications is that they are **stateless**.

Each execution is treated independently. A function should never assume that information from a previous request is still available. Any data that needs to persist must be stored using external services such as Amazon S3 or a database.

This design allows AWS to execute multiple requests simultaneously without requiring applications to maintain long-running processes.

Application in this Project

The Feedback API was implemented entirely using a serverless architecture.

Whenever a user submitted or retrieved feedback, Amazon API Gateway invoked an AWS Lambda function. After processing the request and interacting with Amazon S3, the execution environment was automatically terminated until another request arrived.

This demonstrated how serverless architectures simplify backend development while providing automatic scalability and reducing operational overhead.

AWS LAMBDA

After understanding the fundamentals of serverless computing, the next concept introduced was **AWS Lambda**.

AWS Lambda is Amazon's serverless compute service. It allows developers to execute backend code without provisioning or maintaining servers.

Instead of deploying an entire application, developers upload individual functions that execute whenever an event occurs.

Event

|



AWS Lambda

|



Business Logic

|



Response

Lambda Function Structure

Every Lambda function begins with a **handler**.

The handler acts as the entry point that AWS executes whenever the function is invoked.

```
def lambda_handler(event, context):
```

The handler receives two parameters.

```
def lambda_handler(event, context):  
  
    method = event["requestContext"]["http"]["method"]  
  
    if method == "POST":  
  
        return submit_feedback(event)  
  
    elif method == "GET":  
  
        return get_feedback(event)
```

Although the project contained only two API endpoints, routing requests based on the HTTP method inside a single Lambda function simplified the implementation while demonstrating how one function can serve multiple API routes.

Event

The **event** contains all information associated with the request.

For API Gateway, this typically includes:

- HTTP method
- Request body
- Request headers
- Query parameters
- Authentication information

Understanding the event object is important because every client request reaches Lambda through this structure.

Environment Variables

Applications often require configuration values such as bucket names or API endpoints.

Rather than hardcoding these values, Lambda provides **Environment Variables**.

```
BUCKET_NAME = os.environ["STORAGE_BUCKET"]
```

This separates configuration from application logic and allows Terraform to inject infrastructure values during deployment.

IAM Execution Role

Lambda functions interact with other AWS services through an **IAM Execution Role**.

Lambda

|



IAM Role

|



Amazon S3

The execution role defines exactly what actions the function is allowed to perform.

For this project, the role allowed the Lambda function to upload, retrieve, and list feedback objects stored in Amazon S3.

Application in this Project

AWS Lambda formed the backend of the Feedback API.

The Lambda function validated incoming requests, generated timestamps and unique identifiers, stored feedback in Amazon S3, and retrieved feedback whenever requested.

As authentication was introduced later, the same Lambda function also processed authenticated requests received through API Gateway.

AMAZON API GATEWAY

While AWS Lambda provides the backend compute required to execute application logic, it cannot receive HTTP requests directly from users. A mechanism is required to expose Lambda functions as web APIs that can be accessed securely over the internet.

This responsibility is handled by Amazon API Gateway.

API Gateway acts as the entry point for serverless applications. It receives client requests, forwards them to the appropriate backend service, and returns the response to the client.

The communication flow is shown below.

Client

|

HTTP Request

|



API Gateway

|



AWS Lambda

|



HTTP Response

Separating request handling from backend logic simplifies application development. Lambda focuses only on processing business logic, while API Gateway manages networking and client communication.

```
return {  
  
    "statusCode": 200,  
  
    "headers": {  
  
        "Content-Type": "application/json"  
  
    },  
  
    "body": json.dumps(response)  
  
}
```

REST APIs

The Feedback API follows the principles of a REST (Representational State Transfer) API.

REST is one of the most commonly used architectural styles for designing web services. It uses standard HTTP methods to perform operations on resources.

Some commonly used HTTP methods are:

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Update existing data

DELETE	Remove existing data
--------	----------------------

For this project, only GET and POST requests were required.

- POST was used to submit new feedback.
- GET was used to retrieve previously submitted feedback.

Using standard HTTP methods makes APIs easier to understand and maintain.

Request–Response Lifecycle

Every API request follows a common lifecycle.

Client

|



API Gateway

|



Lambda Function

|



Business Logic

|



Response

|



Client

The client sends a request containing:

- HTTP method
- Headers
- Request body
- Query parameters

API Gateway converts this request into an event object and forwards it to Lambda.

After processing the request, Lambda returns a response containing a status code and response body. API Gateway then converts this back into a standard HTTP response before returning it to the client.

HTTP Status Codes

An API should communicate the outcome of every request clearly.

Rather than simply returning data, APIs use HTTP Status Codes to indicate whether an operation succeeded or failed.

Some commonly used status codes are:

Status Code	Meaning
200	Request completed successfully
201	Resource created successfully
400	Invalid request
401	Authentication required
403	Permission denied
404	Resource not found

500	Internal server error
-----	-----------------------

Returning meaningful status codes simplifies debugging while allowing frontend applications to respond appropriately to different situations.

Cross-Origin Resource Sharing (CORS)

The frontend and backend of a web application are often hosted on different domains.

Modern browsers block such requests unless the server explicitly allows them through Cross-Origin Resource Sharing (CORS).

For this reason, API Gateway was configured with an appropriate CORS policy, allowing the CloudFront-hosted frontend to communicate securely with the backend API.

Although CORS appears to be a browser feature, it plays an important role in protecting users from malicious cross-origin requests.

Application in this Project

Amazon API Gateway acted as the communication layer between the frontend application and AWS Lambda.

The API exposed two primary endpoints.

Endpoint	Purpose
POST /feedback	Submit feedback
GET /feedback	Retrieve feedback

As the project evolved, API Gateway was also configured with a JWT Authorizer, allowing only authenticated users to access protected endpoints while rejecting unauthorized requests before they reached the backend.

AMAZON COGNITO

As the Feedback API evolved, protecting backend endpoints became an important requirement. Public APIs are suitable for demonstrations, but real-world applications require mechanisms to verify user identity before granting access to protected resources.

This responsibility is handled by Amazon Cognito.

Amazon Cognito is AWS's managed identity service. It provides secure user authentication without requiring developers to build their own login system.

Authentication

Authentication is the process of verifying a user's identity.

It answers one simple question.

Who is making this request?

Only after a user's identity has been verified can the application determine what resources they are allowed to access.

Amazon Cognito User Pools

Cognito stores users inside a User Pool.

A User Pool acts as a managed directory containing registered users and their associated information.

Each user record may include:

- Email address
- Verification status
- User groups
- User attributes

Rather than maintaining user information within the application, Cognito becomes the central source of identity.

Cognito Hosted UI

One feature that significantly simplified development was the Hosted UI.

Instead of creating login and registration pages manually, Cognito provides a secure authentication interface.

The Hosted UI supports:

- User registration
- User login
- Email verification
- Password reset

Users are simply redirected to Cognito whenever authentication is required.

OAuth 2.0 Authorization Code Flow

After successful login, Cognito does not immediately return an Access Token.

Instead, the application first receives an Authorization Code, which is exchanged for authentication tokens.

The authentication flow is shown below.

User

|



Cognito Hosted UI

|



User Login

|



Authorization Code

|



Access Token

|



Application

This approach improves security because user credentials are handled entirely by Cognito.

JSON Web Tokens (JWT)

After authentication, Cognito issues JSON Web Tokens (JWTs).

The Feedback API primarily uses the Access Token, which is included in every protected API request.

Authorization:

Bearer <Access Token>

API Gateway validates the token before allowing the request to reach the backend.

Authentication vs Authorization

Authentication verifies who the user is.

Authorization determines what the authenticated user is allowed to access.

This distinction became particularly important while implementing Role-Based Access Control using Cognito User Groups.

Application in this Project

Amazon Cognito provided authentication for the Feedback API.

Users authenticated through the Hosted UI before accessing protected endpoints. After successful login, an Access Token was attached to all protected requests, while Cognito User Groups distinguished between standard users and administrators.

AMAZON CLOUDWATCH

Building cloud applications is only one part of the development process. Once an application has been deployed, it becomes equally important to monitor its behaviour, identify failures, and troubleshoot unexpected issues.

AWS provides these capabilities through Amazon CloudWatch.

CloudWatch is AWS's monitoring and observability service. It collects logs, metrics, and events from AWS resources, allowing developers to monitor applications after deployment.

Although CloudWatch offers many monitoring features, the primary feature used during this project was CloudWatch Logs.

CloudWatch Logs

Whenever a Lambda function executes, anything written using Python's `print()` function is automatically sent to CloudWatch. Instead of storing log files locally, AWS organises logs into Log Groups and Log Streams.

CloudWatch



Log Group



Log Stream



Execution Logs

A Log Group contains all logs generated by a particular AWS resource.

Each execution environment within that resource creates its own Log Stream, making it easier to inspect individual Lambda executions.

Importance of Logging

Logging became one of the most useful debugging tools during development.

Printing the incoming event allowed the complete API Gateway request to be inspected.

```
print(event)
```

As authentication was introduced, CloudWatch was also used to inspect JWT claims, verify Cognito group memberships, and debug Role-Based Access Control.

Without centralized logging, debugging distributed cloud applications becomes significantly more difficult because developers do not have direct access to the underlying servers.

Application in this Project

CloudWatch was used throughout the implementation of the Feedback API to monitor Lambda executions and troubleshoot application behaviour.

It was particularly useful for:

- Inspecting API Gateway requests.
- Debugging Lambda execution.
- Verifying Role-Based Access Control.
- Monitoring interactions with Amazon S3.

CloudWatch became the primary debugging tool throughout the project. During development, it was used to inspect incoming API Gateway events, verify request payloads, monitor Lambda execution, and troubleshoot interactions with Amazon S3. Having centralized logs made it much easier to identify issues without needing direct access to the execution environment.

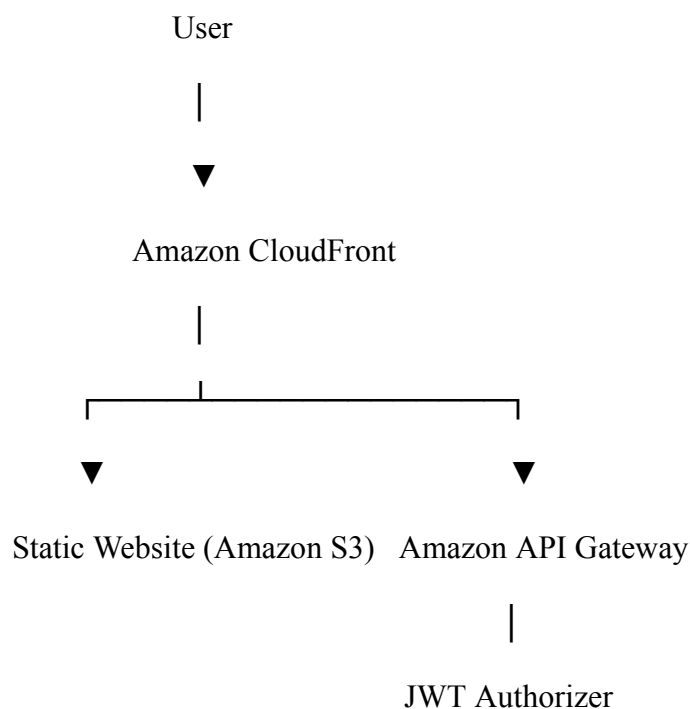
3. PROJECT IMPLEMENTATION

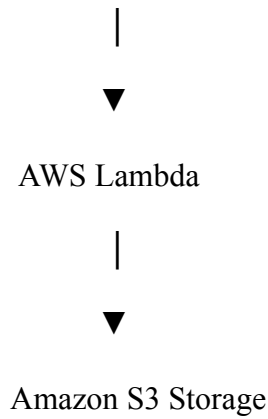
After understanding the individual AWS services introduced throughout the week, the final step was integrating them into a complete serverless application.

Rather than relying on a single backend server, the application was designed by combining multiple managed AWS services. Each service performs one specialised responsibility while communicating with the others to complete a request.

The final architecture is shown below.

OVERALL ARCHITECTURE





The architecture demonstrates an important principle of cloud-native development.

Rather than building a monolithic application, functionality is distributed across specialised AWS services. This improves scalability, maintainability, and separation of concerns.

FEEDBACK STORAGE DESIGN

Instead of storing all feedback inside a single file, each submission was stored as an independent JSON object in Amazon S3.

Example structure:

feedback-storage/

|— 8f8d6....json

|— c713e....json

|— 5e93a....json

Each object contained:

```
{  
  "id": "...",  
  "name": "...",  
  "feedback": "...",  
  "timestamp": "..."  
}
```

Why this approach?

- Avoids file conflicts.
- Makes every submission independent.
- Simplifies writing new feedback.
- Demonstrates object-based storage.

INFRASTRUCTURE PROVISIONING

As in Week 1, every AWS resource was provisioned using Terraform.

The infrastructure included:

- Amazon S3 buckets
- Amazon CloudFront Distribution
- Origin Access Control (OAC)
- AWS Lambda
- Amazon API Gateway
- IAM Roles and Policies
- Amazon Cognito
- Cognito User Groups
- JWT Authorizer

Deployment Challenge

While deploying the infrastructure using Terraform, the deployment initially failed when Terraform attempted to access the Lambda function.

The error returned by AWS was:

AccessDeniedException

lambda:GetFunction

The issue was caused by the IAM user used for Terraform deployment not having permission to perform the **lambda:GetFunction** action.

Updating the IAM policy with the required Lambda permissions resolved the issue and allowed Terraform to successfully provision the remaining resources.

Although the issue was resolved with a relatively small policy change, it demonstrated an important aspect of Infrastructure as Code. Terraform not only creates resources but also reads and manages their state, meaning the deployment identity must have permissions for both provisioning and inspecting AWS resources.

AUTHENTICATION WORKFLOW

User authentication was performed using Amazon Cognito Hosted UI.

Rather than implementing login pages manually, authentication was delegated entirely to Cognito.

The authentication flow is shown below.

User

|



CloudFront Website

|



Cognito Hosted UI

|



User Login

|



Authorization Code

|



Access Token

|



Frontend

Once authentication completed successfully, the frontend stored the Access Token and included it in all protected API requests.

FEEDBACK SUBMISSION WORKFLOW

Submitting feedback involved communication between several AWS services.

User



HTML Form



JavaScript



POST /feedback



API Gateway



Lambda



Amazon S3

The Lambda function validated the request, generated a unique identifier and timestamp, and stored the feedback as an individual JSON object in Amazon S3.

Each submission was stored independently, preventing filename collisions while simplifying retrieval.

FRONTEND VALIDATION

During testing, an unexpected issue was observed where newly submitted feedback contained leading whitespace even when the user had not entered any extra spaces.

After inspecting the HTML structure, the issue was traced to the formatting of the `<textarea>` element. The indentation between the opening and closing tags became part of the textarea's initial value and was therefore submitted along with the feedback.

Original implementation:

```
<textarea  
  
    id="feedback"  
  
    placeholder="Enter your feedback">  
  
</textarea>
```

Updated implementation:

```
<textarea  
  
    id="feedback"  
  
    placeholder="Enter your feedback"></textarea>
```

To further improve input validation, JavaScript's `.trim()` method was also used before sending requests to the API.

```
const feedback =  
  
    document.getElementById("feedback").value.trim();
```

Although this was a small issue, it reinforced the importance of validating user input at the frontend before processing it in the backend.

FEEDBACK RETRIEVAL WORKFLOW

Retrieving feedback required the Lambda function to aggregate multiple JSON objects stored in Amazon S3 before returning them as a single response.

The workflow is illustrated below.

User

|



GET /feedback

|



API Gateway

|



JWT Validation (Protected Endpoints)

|



AWS Lambda

|

├── ListObjects

|

├── Read each JSON object

|

├── Deserialize JSON

|

└── Combine into a single response

|



Return JSON Array

Unlike a traditional database query, Amazon S3 stores each feedback submission as an independent object. Retrieving all feedback therefore required several processing steps inside the Lambda function.

The Lambda function:

- Retrieves the list of feedback objects stored in the S3 bucket.
- Reads each JSON object individually.
- Converts the object contents into Python dictionaries.
- Aggregates all feedback entries into a single list.
- Returns the combined response through API Gateway.

This approach worked well for the scale of the project while demonstrating how object storage can be used as a simple backend datastore. It also highlighted one of the limitations of using Amazon S3 for structured application data, as retrieving multiple records requires reading each object individually rather than performing database queries.

ROLE-BASED ACCESS CONTROL

After authentication was implemented, the application was extended to support multiple permission levels.

Three categories of users were supported.

User Type	Permissions
Guest	Submit anonymous feedback
User	Submit feedback and retrieve anonymized feedback
Administrator	Submit feedback and retrieve feedback with user identities

API Gateway validates the user's JWT before invoking Lambda.

The Lambda function then inspects the user's Cognito group membership and determines what information should be returned.

This approach ensures that authorization decisions are enforced by the backend rather than relying solely on frontend controls.

4. KEY LEARNINGS & TAKEAWAYS

Week 2 focused on understanding how multiple AWS services can be integrated to build a secure serverless application. Rather than learning individual services in isolation, the project demonstrated how each service contributes to the overall architecture.

Some of the key takeaways from this week are:

- Serverless applications execute only when triggered by events, reducing infrastructure management and operational costs.
- AWS Lambda enables backend applications to be built without provisioning or maintaining servers.
- Amazon API Gateway provides a secure interface between clients and backend services while managing request routing.
- Amazon Cognito simplifies authentication by securely managing user identities and issuing JWT tokens.
- Authentication verifies user identity, while authorization determines what resources an authenticated user can access.
- Role-Based Access Control (RBAC) allows different users to access different levels of functionality through Cognito User Groups.
- Amazon CloudWatch provides centralized logging for monitoring and debugging serverless applications.
- Infrastructure as Code enables cloud resources to be provisioned consistently using Terraform.
- Modern cloud applications are built by integrating multiple specialised AWS services, each performing a well-defined responsibility.

This week established the foundation for developing secure cloud-native applications and demonstrated how AWS services can be combined to build scalable, maintainable, and production-oriented serverless architectures.